

QUESTIONS 19 / 20 completed

Bank

BANK

Write a Java program to implement a basic banking system using inheritance. Create a parent class called Account with attributes such as account number, account holder name, and balance. Then, create child classes such as SavingsAccount and CheckingAccount that inherit from the Account class and have their own unique methods for withdrawing and depositing money.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         List<Account> accounts = new ArrayList<>();
6         accounts.add(new SavingsAccount("123456789", "Jane Doe", 1.5, 5000));
7         accounts.add(new CheckingAccount("987654321", "John Smith", 1000, 1000));
8         System.out.println("The accounts in the system are:");
9         for (int i = 0; i < accounts.size(); i++) {
10             System.out.println((i + 1) + ". " + accounts.get(i).getAccountInfo());
11         }
12         for (int i = 0; i < 2; i++) {
13             int idx = sc.nextInt() - 1;
14             String type = sc.next();
15             double amt = sc.nextDouble();
16             Account acc = accounts.get(idx);
17             if (type.equalsIgnoreCase("deposit")) {
18                 acc.deposit(amt);
19             } else {
20                 acc.withdraw(amt);
21             }
22         }
23     }
24 }
```

OUTPUT

```
The accounts in the system are:
1. Savings Account (123456789) - Jane Doe (Balance: $5000.00, Interest Rate: 1.5%)
2. Checking Account (987654321) - John Smith (Balance: $1000.00, Overdraft Limit: $1000.00)

Transaction successful. New balance: $7500.00

Transaction successful. New balance: $-1000.00 (overdraft limit reached)
```

INPUT

```
1
deposit
2500
2
withdraw
2000
```

QUESTIONS 19 / 20 completed

Employee ▾

EMPLOYEE

Write a Java program to implement a basic employee management system using inheritance. Create a parent class called Employee with attributes such as name, id, and salary. Then, create child classes such as Manager and SalesPerson that inherit from the Employee class and have their own unique methods for managing teams and making sales.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class EmployeeManagementSystem {
2     public static void main(String[] args) {
3         Manager mgr = new Manager("Alice Smith", 101, 75000.0);
4         mgr.displayDetails();
5         mgr.manageTeam();
6         System.out.println("-----");
7         SalesPerson sp = new SalesPerson("Bob Jones", 102, 45000.0);
8         sp.displayDetails();
9         sp.makeSale();
10    }
11 }
12 class Employee {
13     String name;
14     int id;
15     double salary;
16     Employee(String name, int id, double salary) {
17         this.name = name;
18         this.id = id;
19         this.salary = salary;
20     }
21     void displayDetails() {
```

OUTPUT

```
Employee Name: Alice Smith
Employee ID: 101
Salary: $75000.0
Alice Smith is managing the team and overseeing projects.
-----
Employee Name: Bob Jones
Employee ID: 102
Salary: $45000.0
Bob Jones is making a sale and interacting with customers.
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Animal-1 ▾

ANIMAL-1

Write a Java program to implement a basic animal classification system using inheritance. Create a parent class called Animal with attributes such as name and habitat. Then, create child classes such as Mammal, Reptile, and Bird that inherit from the Animal class and have their own unique methods for eating and reproducing.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Mammal lion = new Mammal("Lion", "Savannah");
4         lion.displayInfo();
5         lion.eat();
6         lion.reproduce();
7         System.out.println();
8         Reptile python = new Reptile("Python", "Forest");
9         python.displayInfo();
10        python.eat();
11        python.reproduce();
12        System.out.println();
13        Bird eagle = new Bird("Eagle", "Mountains");
14        eagle.displayInfo();
15        eagle.eat();
16        eagle.reproduce();
17    }
18 }
19 class Animal {
20     String name;
21     String habitat;
```

OUTPUT

```
Name: Lion
Habitat: Savannah
Lion eats milk or plants.
Lion gives birth to live young.

Name: Python
Habitat: Forest
Python eats insects or small animals.
Python lays eggs.
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Shapes-1 ▾

SHAPES-1

Write a Java program to implement a basic shape hierarchy using inheritance. Create a parent class called Shape with attributes such as color and area. Then, create child classes such as Circle, Square, and Triangle that inherit from the Shape class and have their own unique methods for calculating area and perimeter.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("--- Circle ---");
4         Circle circle = new Circle("Red", 5.0);
5         circle.calculateArea();
6         circle.displayShapeInfo();
7         circle.calculatePerimeter();
8         System.out.println("\n--- Square ---");
9         Square square = new Square("Blue", 4.0);
10        square.calculateArea();
11        square.displayShapeInfo();
12        square.calculatePerimeter();
13        System.out.println("\n--- Triangle ---");
14        Triangle triangle = new Triangle("Green", 3.0, 4.0, 3.0, 4.0, 5.0);
15        triangle.calculateArea();
16        triangle.displayShapeInfo();
17        triangle.calculatePerimeter();
18    }
19 }
20 class Shape {
21     String color;
```

OUTPUT

```
--- Circle ---
Color: Red
Area: 78.53981633974483
Circle Perimeter: 31.41592653589793

--- Square ---
Color: Blue
Area: 16.0
Square Perimeter: 16.0
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Library ▾

LIBRARY

Write a Java program to implement a basic library management system using inheritance. Create a parent class called Item with attributes such as title and author. Then, create child classes such as Book and DVD that inherit from the Item class and have their own unique attributes and methods for borrowing and returning items.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     static class Item {
3         String title, author;
4         boolean borrowed = false;
5         Item(String t, String a) { title = t; author = a; }
6         void borrow() { if (!borrowed) borrowed = true; }
7         void returnItem() { if (borrowed) borrowed = false; }
8         public String toString() { return title + " - " + (borrowed ? "Borrowed"
9     }
10    static class Book extends Item {
11        String isbn;
12        Book(String t, String a, String i) { super(t, a); isbn = i; }
13        void borrowBook() { borrow(); }
14        void returnBook() { returnItem(); }
15    }
16    static class DVD extends Item {
17        int duration;
18        DVD(String t, String a, int d) { super(t, a); duration = d; }
19        void borrowDVD() { borrow(); }
20        void returnDVD() { returnItem(); }
21    }
```

OUTPUT

```
The Alchemist - Borrowed
Inception - Borrowed
The Alchemist - Available
Inception - Available
```

INPUT

```
The Alchemist
Paulo Coelho
ISBN1
Inception
Nolan
148
```

QUESTIONS 19 / 20 completed

University

UNIVERSITY

Write a Java program to implement a basic university management system using inheritance. Create a parent class called Person with attributes such as name and age. Then, create child classes such as Student and Professor that inherit from the Person class and have their own unique attributes and methods for managing coursework and teaching classes.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         Student s = new Student(sc.nextLine(), sc.nextInt(), sc.next(), sc.next());
6         System.out.println("Student Info:");
7         s.display();
8         s.manageCoursework();
9         sc.nextLine();
10        Professor p = new Professor(sc.nextLine(), sc.nextInt(), sc.next(), sc.next());
11        System.out.println("\nProfessor Info:");
12        p.display();
13        p.teachClass();
14    }
15 }
16 class Person {
17     String name;
18     int age;
19     Person(String n, int a) { name = n; age = a; }
20     void display() { System.out.println("Name: " + name + ", Age: " + age); }
21 }
```

OUTPUT

```
Student Info:
Name: Asha, Age: 20
ID: S101
Managing course: BCA

Professor Info:
Name: Dr.Ravi, Age: 45
ID: P201
Teaching: OOP
```

INPUT

```
Asha
20
S101
BCA
Dr.Ravi
45
P201
OOP
```


QUESTIONS 19 / 20 completed

Games ▾

GAMES

Write a Java program to implement a basic game system using inheritance. Create a parent class called Game with attributes such as title and genre. Then, create child classes such as ActionGame and PuzzleGame that inherit from the Game class and have their own unique attributes and methods for playing the game.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     static class Game {
3         String title, genre;
4         Game(String t, String g) { title = t; genre = g; }
5         void play() { System.out.println("Playing " + title + " (" + genre + ")")
6     }
7     static class ActionGame extends Game {
8         int difficulty;
9         ActionGame(String t, String g, int d) { super(t, g); difficulty = d; }
10        @Override void play() {
11            System.out.println("Action: " + title + " | Difficulty: " + difficulty);
12        }
13    }
14    static class PuzzleGame extends Game {
15        int levels;
16        PuzzleGame(String t, String g, int l) { super(t, g); levels = l; }
17        @Override void play() {
18            System.out.println("Puzzle: " + title + " | Levels: " + levels);
19        }
20    }
21    public static void main(String[] args) {
```

OUTPUT

```
Action: Street Fighter | Difficulty: 7
Puzzle: Mind Breaker | Levels: 25
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Resturant ▾

RESTURANT

Write a Java program to implement a basic restaurant management system using inheritance. Create a parent class called Menu with attributes such as name and price. Then, create child classes such as Appetizer and Entree that inherit from the Menu class and have their own unique attributes and methods for ordering and serving food.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     static class Menu {
3         String name; double price;
4         Menu(String n, double p){ name=n; price=p; }
5     }
6     static class Appetizer extends Menu {
7         String flavor;
8         Appetizer(String n, double p, String f){ super(n,p); flavor=f; }
9         void order(){ System.out.println("Appetizer ordered: "+name+" ("+flavor+"
10        void serve(){ System.out.println("Appetizer served: "+name); }
11    }
12    static class Entree extends Menu {
13        String ingredient;
14        Entree(String n, double p, String i){ super(n,p); ingredient=i; }
15        void order(){ System.out.println("Entree ordered: "+name+" ["+ingredient+"
16        void serve(){ System.out.println("Entree served: "+name); }
17    }
18    public static void main(String[] args) {
19        Appetizer a = new Appetizer("Garlic Bread", 5.99, "Savory");
20        Entree e = new Entree("Grilled Salmon", 18.50, "Fresh Salmon");
21        a.order(); a.serve();
22    }
23 }
```

OUTPUT

```
Appetizer ordered: Garlic Bread (Savory)
Appetizer served: Garlic Bread
Entree ordered: Grilled Salmon [Fresh Salmon]
Entree served: Grilled Salmon
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Forecasting ▾

FORECASTING

Write a Java program to implement a basic weather forecasting system using inheritance. Create a parent class called Forecast with attributes such as date and temperature. Then, create child classes such as SunnyForecast and RainyForecast that inherit from the Forecast class and have their own unique attributes and methods for predicting the weather.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         SunnyForecast sunny = new SunnyForecast("2026-08-31", 30.5, 6.0);
4         sunny.predict();
5         System.out.println();
6         RainyForecast rainy = new RainyForecast("2026-08-31", 26.0, 12.5, true);
7         rainy.predict();
8     }
9 }
10 class Forecast {
11     String date;
12     double temperature;
13     Forecast(String date, double temperature) {
14         this.date = date;
15         this.temperature = temperature;
16     }
17     void displayBasicInfo() {
18         System.out.println("Date: " + date);
19         System.out.println("Temperature: " + temperature + "°C");
20     }
21 }
```

OUTPUT

```
System Error: Could not reach compiler or server error occurred.
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Rental ▾

RENTAL

Write a Java program to implement a basic car rental system using inheritance. Create a parent class called Vehicle with attributes such as make and model. Then, create child classes such as Sedan and SUV that inherit from the Vehicle class and have their own unique attributes and methods for renting and returning cars.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Sedan mySedan = new Sedan("Toyota", "Camry", 5);
4         SUV mySUV = new SUV("Jeep", "Wrangler", true);
5         System.out.println("--- Car Rental System ---");
6         mySedan.showDetails();
7         mySedan.rent();
8         mySedan.rent();
9         mySedan.returnCar();
10        System.out.println();
11        mySUV.showDetails();
12        mySUV.rent();
13        mySUV.returnCar();
14        mySUV.returnCar();
15    }
16 }
17 class Vehicle {
18     protected String make;
19     protected String model;
20     public Vehicle(String make, String model) {
21         this.make = make;
```

OUTPUT

```
--- Car Rental System ---
Make: Toyota, Model: Camry, Seating Capacity: 5
Sedan Toyota Camry has been rented successfully.
Sorry, this Sedan is already rented.
Sedan Toyota Camry has been returned.

Make: Jeep, Model: Wrangler, 4WD: Yes
SUV Jeep Wrangler has been rented successfully.
SUV Jeep Wrangler has been returned.
This SUV was not rented.
```

INPUT

Your INPUT goes
here!